

SIEMENS EDA

Algorithmic C (AC) Floating-Point Math Library Reference Manual

Software Version v3.8.1
May 2025



Copyright 2025 Siemens

This document contains information that is proprietary to Siemens. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

Table of Contents

Chapter 1: Introduction to ac_fpmath..... 2

1.1. Summary of Classes and Functions..... 2

1.1.1. *Block Floating-Point*..... 2

1.1.2. *Miscellaneous Floating-Point Math Functions*..... 2

1.2. Using the ac_fpmath library..... 2

Chapter 2: Block Floating-Point (ac_block_float)..... 3

2.1. The structure of ac_block_float..... 3

2.2. Conversion between ac_std_float and ac_block_float..... 5

2.3. Arithmetic Operations..... 6

2.3.1. *Addition*..... 6

2.4. Arithmetic Binary Operators..... 7

2.5. Arithmetic Assign Operators..... 7

2.6. Current limitations..... 7

Chapter 3: Miscellaneous ac_fpmath Functions..... 9

3.1. Standard Floating-Point Square (ac_sqr)..... 9

3.2. Standard Floating-Point Dot Product Functions (fdot_tree / fdot_tree_ptr)..... 10

3.2.1. *Function Outline*..... 10

3.2.2. *Special Values*..... 12

3.3. Standard Floating-Point Dual-Path Addition/Subtraction (ac_add_dp/ ac_sub_dp)..... 13

3.3.1. *The structure of a dual-path adder*..... 14

Chapter 1: Introduction to ac_fpmath

The Algorithmic C Floating-Point Math Library (*ac_fpmath*) library is a member of the *ac_math* library and contains synthesizable C++ functions designed for floating-point operations.

The following sections provide a summary of the *ac_fpmath* library:

- [Summary of Classes and Functions](#)
- [Using the ac_fpmath library](#)

1.1. Summary of Classes and Functions

This following sections summarizes the functions and classes currently supported in the *ac_fpmath* library.

1.1.1. Block Floating-Point

The library provides the *ac_block_float* class that allows the implementation of floating-point addition with decreased delay. The *ac_block_float* class is based on the standard floating-point library (*ac_std_float*).

1.1.2. Miscellaneous Floating-Point Math Functions

A number of floating-point math functions that are also provided. These are the square, the dual-path addition, and the dot product.

1.2. Using the ac_fpmath library

The hierarchy of the contained library directories is shown here:

```
.
|-- include
|   |-- ac_fpmath_int
|   |   |-- ac_block_float.h
|   |   |-- ac_std_float_add_dualpath.h
|   |   |-- ac_std_float_square.h
|   |   |-- ac_std_float_dot_prod.h
```

The library is packaged within Catapult HLS and is installed in `$MGC_HOME/share/include`.

Chapter 2: Block Floating-Point (ac_block_float)

The block floating-point datatype (ac_block_float) is an alternative way to represent standard floating-point (ac_std_float) values, and so it has been developed based on the ac_std_float datatype. Its purpose is to enable low latency addition/subtraction operations for the ac_std_float datatype. Being completely dependent to the ac_std_float, the structure of the datatype derives both from the corresponding ac_std_float datatype and some extra parameters. The new ac_block_float parameters are the BlockE and the ExtraLSBs. The ac_block_float class is organized as presented in Table 1.

Table 1: The ac_block_float class parameters.

Code	Description
<pre>template< class T, int BlockE, int ExtraLSBs > ac_block_float {};</pre>	T can be any ac_std_float<W, E> and defines the standard float datatype from which the ac_block_float derives. [1] BlockE specifies the size of the block exponent field in the ac_block_float datatype. The value can be a number smaller than the size of the dependent ac_std_float datatype. [2] ExtraLSBs specifies the width of the last segment of the block fraction. The value of this parameter affects the accuracy of the result. [3]

[1] In the current version, there is a limitation of the supported ac_std_float datatypes. See [current limitations](#).

[2] In this version of the ac_block_float, the supported values are limited. See [current limitations](#).

[3] See [the structure of ac_block_float](#) for more details.

The following list shows the topics that are discussed in the remainder of this chapter:

- [The structure of ac_block_float](#)
- [Conversion between ac_std_float and ac_block_float](#)
- [Arithmetic Operations](#)
- [Arithmetic Binary Operators](#)
- [Arithmetic Assign Operators](#)
- [Current limitations](#)

2.1. The structure of ac_block_float

The ac_block_float, unlike the ac_std_float, is composed of two fields: the block exponent and the block fraction. These two fields serve similar purposes as the ac_std_float's exponent and mantissa fields.

The block exponent, exactly like the `ac_std_float` exponent, stores the exponent of the datatype. The size of the block exponent is defined through the `BlockE` parameter. As the `ac_block_float` is highly dependent to an `ac_std_float` datatype, `BlockE` should have a value that is in the range of (1, E), where E is the size of the `ac_std_float`'s exponent. The block exponent stores the `BlockE` number of MSBs of the corresponding `ac_std_float` exponent, while the remaining LSBs of the exponent are used for mantissa alignment (see [Conversion between `ac\_std\_float` and `ac\_block\_float`](#)).

The block fraction, unlike the `ac_std_float` mantissa, follows a 2's complement representation. In addition, the hidden bit is not omitted, but it is present in the block fraction. The size of the block fraction depends in the `ac_std_float` datatype defined through the template parameter T, as well as on both `BlockE` and `ExtraLSBs` parameters. The block fraction can be broken down into three segments as follows:

- **Block alignment bits:** When an `ac_std_float` is converted to `ac_block_float`, the mantissa together with the hidden bit should be aligned, depending on the LSB part of the exponent that is not stored in the block exponent field (see [Conversion between `ac\_std\_float` and `ac\_block\_float`](#)). The size of this part of the block fraction should be enough so that the mantissa can be shifted to the maximum value without losing any bit. The width of the LSB part of the `ac_std_float` exponent that will be used for shifting is (E - `BlockE`). The maximum shifting amount would be $2^{E-\text{BlockE}}$ and that is the size of the block alignment segment of the block fraction.
- **Precision bits:** The precision segment is the LSB part of the block fraction. The size of this segment is controlled by the `ExtraLSBs` parameter. When aligning two numbers for addition, one can be shifted to the right. Keeping the shifted bits will increase the accuracy of the result when it will be converted back to `ac_std_float`. As a result, the size of this segment affects the accuracy of the result.
- Between the block alignment bits and the precision bits, there is a segment with size equal to the `ac_std_float` mantissa plus the hidden bit.

Finally, one more bit for overflow protection is used, leading to a total size of $(2 + 2^{E-\text{BlockE}} + (M+1) + \text{ExtraLSBs})$ bits, where M is the size of the `ac_std_float` mantissa. Figure 1 outlines the structure of the `ac_block_float` and how it derives from the respective standard floating-point datatype.

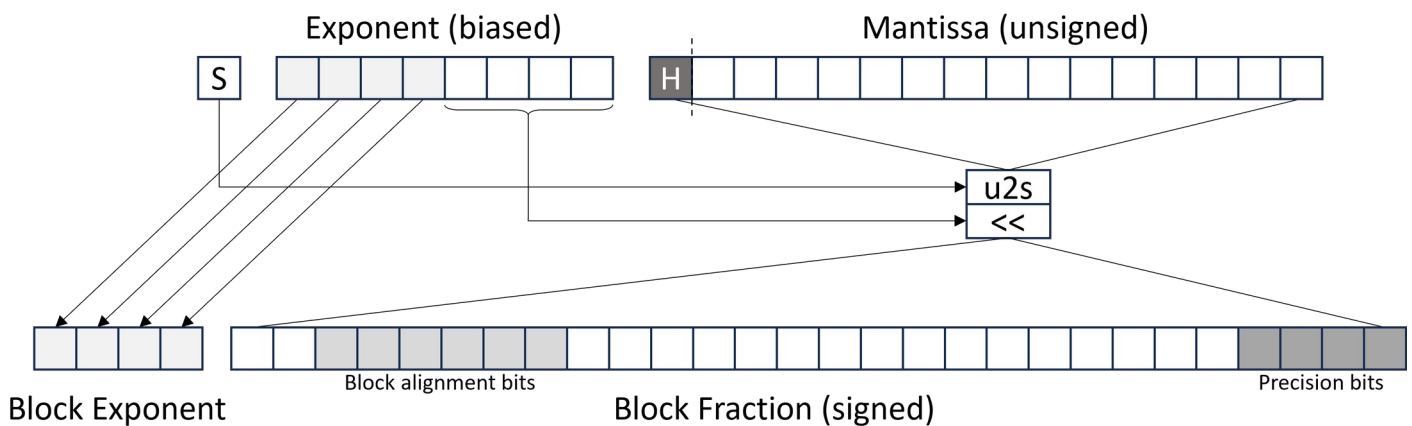


Figure 1: The structure of the `ac_block_float` datatype with respect to the `ac_std_float`.

Apart from the block exponent and the block fraction, the `ac_block_float` stores all the flags for special values (Infinite, Zero, Normal, NaN).

2.2. Conversion between `ac_std_float` and `ac_block_float`

The `ac_block_float` supports conversion from and to `ac_std_float`. The summary of the functions is presented in Table 2.

Table 2: Summary of the `ac_block_float` conversion functions.

Code	Description
<pre>template< bool No_SubNormals > ac_block_float<T> from_ac_std_float(const T &op) {} template< ac_q_mode QR, bool No_SubNormals > T to_ac_std_float() const {}</pre>	QR is the rounding mode. Currently the supported modes are the ones supported by the <code>ac_std_float</code> library: <code>AC_RND_CONV</code>, <code>AC_TRN_ZERO</code>, <code>AC_RND_INF</code>, <code>AC_RND_CONV_ODD</code> If <code>No_SubNormals = true</code> then subnormal inputs are treated as zeros and subnormal results are flushed to zeros.

The conversion of an `ac_std_float` to `ac_block_float` is implemented by the `from_ac_std_float` function. During the conversion, the operations performed can be described as follows:

- The width of the block exponent is already defined by the `BlockE` parameter. The block exponent stores the `BlockE` MSBs from the `ac_std_float`'s exponent.
- The mantissa from the `ac_std_float` is appended with the hidden bit and converted to a signed value following the 2's complement representation, based on the value of the sign.
- The value of the remaining LSBs from the exponent (treated as unsigned) will shift left the mantissa with the hidden bit.
- Special values will be extracted and stored.

The conversion of an `ac_block_float` to `ac_std_float` is implemented by the `to_ac_std_float` function. During the conversion, the operations performed can be described as follows:

- Convert the block fraction to unsigned and shift it so that the value will completely move outside (to the right) of the block alignment segment. Keep the shifted-out bits for rounding.
- Set the block exponent as the `BlockE` MSBs of the `ac_std_float` exponent and add the shifting amount computed above.
- Round the result.
- Adjust the mantissa and the exponent and take into account any special values.

An example using these two functions is presented as follows:

```
#include <ac_fpmath_int/ac_block_float.h>

#define NoSubNormals true

typedef ac_std_float<32,8> flt;
typedef ac_math::ac_block_float<flt,4,8> block_flt;

flt op = 8.3;

// convert ac_std_float to ac_block_float
block_flt bop = block_flt::from_ac_std_float< NoSubNormals >(op);

// convert ac_block_float to ac_std_float
flt op2 = bop.to_ac_std_float<AC_RND_CONV, NoSubNormals >()
```

2.3. Arithmetic Operations

Currently the block floating-point datatype supports additions only between two `ac_block_float` operands.

2.3.1. Addition

The implementation of the `ac_block_float` addition is similar to that of the `ac_std_float`, first find the maximum exponent, then align the fractions, perform the addition and normalize the result. Unlike the `ac_std_float`, on the `ac_block_float` addition there is no rounding at the end. Rounding only occurs when converting an `ac_block_float` back to `ac_std_float`.

In `ac_block_float` addition, the alignment before the addition can be implemented as a small multiplexing logic of constant shifting. This is because a difference of 1 between the block exponents of the two addends equals to a shifting by $2E - \text{BlockE}$ bits.

In the current version of the addition, the majority of operations are parallelized, targeting a small delay solution, and so the `ac_block_float` addition makes a good candidate for implementing an accumulator. By utilizing the `ac_block_float` datatype, an accumulator for `ac_std_float` values can be implemented. Here follows an example of an accumulator implementation:

```
#include <ac_fpmath_int/ac_block_float.h>

#define NoSubNormals true

typedef ac_std_float<32,8> flt;
typedef ac_math::ac_block_float<flt,4,8> block_flt;

#pragma hls_pipeline_init_interval 1
void accum_example(const flt &op, flt &out) {
```

```
static block_flt r;
block_flt op1 = block_flt::from_ac_std_float< NoSubNormals >(op);

block_flt op2 = r;
r = op1.add< NoSubNormals >(op2);

out = r.to_ac_std_float<AC_RND_CONV, NoSubNormals >();
}
```

2.4. Arithmetic Binary Operators

The arithmetic binary operator is shown in Table 3.

Table 3: Arithmetic binary operator.

Arithmetic Binary Operator	Description
+	The operator uses AC_RND_CONV as default rounding mode. Subnormal values are supported.

2.5. Arithmetic Assign Operators

The arithmetic assign operator is shown in Table 4.

Table 4: Arithmetic assign operator.

Arithmetic Assign Operator	Description
+=	The operator uses AC_RND_CONV as default rounding mode. Subnormal values are supported.

2.6. Current limitations

In this version of the ac_block_float datatype, the addition operation offers limited implementation solutions, as a result, the only supported configurations are limited to the following:

- Only conversion from 32bit single precision float (ac_std_float<32,8>) is supported.
- The BlockE parameter can take any value between 3 and 7. The ExtraLSBs parameter can take any positive value or zero. However, selecting values for the ExtraLSBs parameter that are not in the ranges presented in the following list, should have no impact on the accuracy of the result.
 - Block Exponent=3, ExtraLSBs= 0 – 32
 - Block Exponent=4, ExtraLSBs= 0 – 16
 - Block Exponent=5, ExtraLSBs= 0 – 8

- Block Exponent=6, ExtraLSBs= 0 – 4
- Block Exponent=7, ExtraLSBs= 0 – 2

NOTE: When the value of ExtraLSBs is less than 2, then the result is no more equivalent to the corresponding ac_std_float addition, as the GRS bits are not kept for rounding.

Chapter 3: Miscellaneous ac_fpmath Functions

The *ac_fpmath* package also provides the following functions:

- [Standard Floating-Point Square \(ac_sqr\)](#)
- [Standard Floating-Point Dot Product Functions \(fdot_tree / fdot_tree_ptr\)](#)
- [Standard Floating-Point Dual-Path Addition/Subtraction \(ac_add_dp/ ac_sub_dp\)](#)

The following subsections describes the implementation and usage of these functions in more detail.

3.1. Standard Floating-Point Square (ac_sqr)

The square function computes the operation a^2 for all the standard floating-point datatypes, namely `ac_std_float`, `ac_ieee_float` and `ac::bfloat16`. The function has only one input argument and returns the result of the square operation in the same datatype as the input. The implementation of the function is contained in the `ac_std_float_square.h` header file.

The square operation can be called by the `ac_sqr` function, which is based on the `ac_std_float` datatype and its implementation is similar to the rest of the `ac_std_float` arithmetic operations. The prototype of the function is given as follows, where the *W* and *E* parameters are inferred by the input datatype:

```
template< ac_q_mode QR=AC_RND_CONV,
          bool No_SubNormals=false, int W, int E >
ac_std_float<W,E> ac_sqr( ac_std_float<W,E> &op )
```

Based on the `ac_std_float`, the function supports the following rounding modes: `AC_RND_CONV`, `AC_TRN_ZERO`, `AC_RND_INF` and `AC_RND_CONV_ODD`. For the `ac_std_float` and `ac_ieee_float` datatypes, the default rounding mode is set to `AC_RND_CONV`, while for the `ac::bfloat16` datatype, the default rounding mode is set to `AC_TRN_ZERO`.

The prototype functions for the `ac_ieee_float` and `ac::bfloat16` datatypes are listed as follows:

```
template< ac_q_mode QR=AC_RND_CONV, bool No_SubNormals=false,
          ac_ieee_float_format Format >
ac_ieee_float<Format> ac_sqr( const ac_ieee_float<Format> &op )

template<ac_q_mode QR=AC_TRN_ZERO, bool No_SubNormals=false >
ac::bfloat16 ac_sqr( const ac::bfloat16 &op )
```

The support for subnormal values can be toggled through the `No_SubNormals` template parameter. By default, the `No_SubNormal` parameter is set to false.

An example of using the square function for all the standard floating-point datatypes is given as follows:

```
#include <ac_std_float.h>
#include <ac_fpmath_int/ac_std_float_square.h>

typedef ac_std_float<32,8> float32;
typedef ac_ieee_float<binary32> ieee32;
typedef ac::bfloat16 bf16;

float32 x1 = 2.0;
float32 r1 = ac_math::ac_sqr(ax1); // default parameters are AC_RND_CONV, false
float32 r2 = ac_math::ac_sqr<AC_TRN_ZERO, true>(x1);

ieee32 x2 = 2.0;
ieee32 r3 = ac_math::ac_sqr(x2); // default parameters are AC_RND_CONV, false
ieee32 r4 = ac_math::ac_sqr<AC_TRN_ZERO, true>(x2);

bf16 x3 = 2.0;
bf16 r5 = ac_math::ac_sqr(x3); // default parameters are AC_TRN_ZERO, false
bf16 r6 = ac_math::ac_sqr<AC_RND_CONV, false>(x3);
```

3.2. Standard Floating-Point Dot Product Functions (fdot_tree / fdot_tree_ptr)

The dot product functions for standard floating-point datatypes provide an implementation of a dot product operation between two input vectors of the same datatype and of arbitrary size. The result of the operation is of the same datatype as the elements of the two input vectors. The two input vectors are provided as arrays of elements which can be passed either as pointers or arrays of references depending on the implementation selected. All the implementations are contained in the header file ac_std_float_dot_prod.h.

3.2.1. Function Outline

The dot product can be called either by the fdot_tree function, which takes two arrays of reference as inputs or the fdot_tree_ptr function that takes two pointers as inputs. Both functions are implemented for all datatypes supported by the ac_std_float.h library, namely ac_std_float, ac_ieee_float and ac::bfloat16. All these different versions are implemented as overload functions which share the same core implementation. The available versions along with their parameters are summarized in Table 5.

Table 5: Summary of the dot product function.

Code	Description
<pre>template< ac_q_mode QR=AC_RND_CONV, bool No_SubNormals=false, int AccExtraLSBs=0, int W, int E,</pre>	W, E define the datatype width and the exponent width of the ac_std_float datatype. The values are being inferred from the input datatype. Format defines the ac_ieee_format in the

```

    int N_ELEMS >
void fdot_tree (
    const ac_std_float<W,E> (&x) [N_ELEMS],
    const ac_std_float<W,E> (&y) [N_ELEMS],
    ac_std_float<W,E> &acc )

template< ac_q_mode QR=AC_RND_CONV,
    bool No_SubNormals=false,
    int AccExtraLSBs=0,
    ac_ieee_float_format Format,
    int N_ELEMS >
void fdot_tree (
    const ac_ieee_float<Format> (&x) [N_ELEMS],
    const ac_ieee_float<Format> (&y) [N_ELEMS],
    ac_ieee_float<Format> &acc )

template< ac_q_mode QR=AC_TRN_ZERO,
    bool No_SubNormals=false,
    int AccExtraLSBs=0,
    int N_ELEMS >
void fdot_tree (
    const ac::bfloat16 (&x) [N_ELEMS],
    const ac::bfloat16 (&y) [N_ELEMS],
    ac::bfloat16 &acc )

template< int N_ELEMS,
    ac_q_mode QR=AC_RND_CONV,
    bool No_SubNormals=false,
    int AccExtraLSBs=0,
    int W, int E >
void fdot_tree_ptr (
    const ac_std_float<W,E> x[N_ELEMS],
    const ac_std_float<W,E> y[N_ELEMS],
    ac_std_float<W,E> &acc )

template< int N_ELEMS,
    ac_q_mode QR=AC_RND_CONV,
    bool No_SubNormals=false,
    int AccExtraLSBs=0,
    ac_ieee_float_format Format >
void fdot_tree_ptr (
    const ac_ieee_float<Format> x[N_ELEMS],
    const ac_ieee_float<Format> y[N_ELEMS],
    ac_ieee_float<Format> &acc )

```

case of ac_ieee_float datatype. Valid values: binary16, binary32, binary64, binary128 and binary256. The value is inferred from the input datatype. N_ELEMS defines the number of elements in each input array. This parameter is required only in the pointer version.

QR is the rounding mode. Currently the supported modes are the ones supported by the ac_std_float library: AC_RND_CONV (default), AC_TRN_ZERO (default for the ac::bfloat16), AC_RND_INF, AC_RND_CONV_ODD.

If No_SubNormals = true then subnormal inputs are treated as zeros and subnormal results are flushed to zeros. The default value is false.

AccExtraLSBs is used to define the number of extra LSB that will be used in the accumulation of the products. This value affects the accuracy of the result. The default value is 0. This parameter can take either positive or negative values.[1]

```
template< int N_ELEMS,
        ac_q_mode QR=AC_TRN_ZERO,
        bool No_SubNormals=false,
        int AccExtraLSBs=0 >
void fdot_tree_ptr (
    const ac::bfloat16 x[N_ELEMS],
    const ac::bfloat16 y[N_ELEMS],
    ac::bfloat16 &acc )
```

[1] Given the width M of the input mantissa, the width of the product of the multiplication is $2M+2$. The accumulator's width is $2M+2+\log_2(N_ELEMS)$. The width of the result of the dot product operation will also be M , as the output datatype follows that of the input. Having already twice the number of required precision bits available, the user can set `AccExtraLSBs` to a negative value, to reduce the size of the accumulator. This reduction in the width of the accumulation may result in accuracy loss for some cases.

The core implementation of the dot product functions performs the multiplication for each pair of inputs in parallel, before adding the products using a tree structured accumulation. Intermediate rounding and normalizations are skipped.

The overall architecture is summarized as follows. First, the special values and the sign, exponent, mantissa part of each input operand are extracted. For each pair of inputs, the signs are being XORed, the exponents are being added, and the mantissas are multiplied. The maximum added exponent is selected and using this as a reference, the products are aligned and converted to 2's complement representation. The aligned products are accumulated in a tree structure of fixed-point additions. The result of the addition gets normalized and rounded, and finally the result is packed as an `ac_std_float` datatype.

The width of the accumulation depends on the width of the products, which is double the width of the input mantissa plus the hidden bit. The width of the accumulation can be adjusted through the `AccExtraLSBs` parameter.

3.2.2. Special Values

As part of the `ac_std_float` library, the inputs and the outputs of the dot product function support special values as defined in the IEEE-754 standard. Each input can have one of the following values: Infinite (positive or negative), Not-A-Number, Subnormal or Normal values. The output result can also have one of these values, which derive from the following scenarios:

- If one or more of the inputs are positive infinite (+Inf), the result is also positive infinite (+Inf).
- If one or more of the inputs are negative infinite (-Inf), the result is also negative infinite (-Inf).
- If there is no infinite input, but the exponent is overflowed to the maximum value, the result is also infinite, having the sign of the computed result.
- In the case of rounding mode `AC_TRN_ZERO`, this value is saturated to the maximum normal number.

- If there are both positive (+Inf) and negative (-Inf) infinite inputs, the result is Not-A-Number.

If one or more inputs are Not-A-Number, the result is also Not-A-Number.

- In the case of No_SubNormals=true, all subnormal input and output values are flushed to zero.

In any other case, the result can be any normal or subnormal value.

NOTE: The dot product function computes a product for each pair of inputs and then accumulates these products. There may be cases where one product would overflow to positive infinite and another product would overflow to negative infinite. In such cases, doing each multiplication separately and then adding the two products, following the IEEE-754 standard, the result would be Not-A-Number. In the case of the dot product function, however, the result will be either positive or negative infinite, depending on the dominant value. When rounding mode is set to AC_TRN_ZERO, the result will be a saturated maximum value instead.

3.3. Standard Floating-Point Dual-Path Addition/Subtraction (ac_add_dp/ ac_sub_dp)

Compared to the add and sub functions of the ac_std_float.h, the dual path architecture for additions and subtractions offers a reduced delay solution for adding or subtracting two standard floating-point datatypes, namely ac_std_float, ac_ieee_float and ac::bfloat16. The ac_add_dp and ac_sub_dp functions have two input arguments and return the result of the addition and subtraction operation respectively, in the same datatype as the inputs. The implementation of these functions are contained in the ac_std_float_add_dualpath.h header file.

The dual path addition/subtraction operation can be called by the ac_add_dp/ac_sub_dp functions, which are based on the ac_std_float datatype and its implementation in similar to the rest of the ac_std_float arithmetic operations. The prototypes of these functions are given as follows, where the W and E parameters are inferred by the input datatype:

```
template<ac_q_mode QR=AC_RND_CONV, bool No_SubNormals=false, int W, int E>
ac_std_float<W,E> ac_add_dp(const ac_std_float<W,E> &op1,
                           const ac_std_float<W,E> &op2)

template<ac_q_mode QR=AC_RND_CONV, bool No_SubNormals=false, int W, int E>
ac_std_float<W,E> ac_sub_dp(const ac_std_float<W,E> &op1,
                           const ac_std_float<W,E> &op2)
```

Based on the ac_std_float, the function supports the following rounding modes: AC_RND_CONV, AC_TRN_ZERO, AC_RND_INF and AC_RND_CONV_ODD. For the ac_std_float and ac_ieee_float datatypes, the default rounding mode is set to AC_RND_CONV, while for the ac::bfloat16 datatype, the default rounding mode is set to AC_TRN_ZERO.

The support for subnormal values can be toggled through the No_SubNormals template parameter. By default, the No_SubNormal parameter is set to false.

An example of using the square function for all the standard floating-point datatypes is given as follows:

```
#include <ac_std_float.h>
#include <ac_fpmath_int/ac_std_float_add_dualpath.h>

typedef ac_std_float<32,8> float32;
typedef ac_ieee_float<binary32> ieeeef32;
typedef ac::bfloat16 bf16;

float32 x1 = 2.0, x2 = 1.5;
float32 r1 = ac_math::ac_sub_dp(x1, x2) //sub, default parameters
                                     // AC_RND_CONV, false
float32 r2 = ac_math::ac_add_dp<AC_TRN_ZERO, true>(x1, x2);

ieeeef32 a1 = 2.0, a2 = 1.5;
ieeeef32 r3 = ac_math::ac_add_dp(a1, a2); //add, default parameters
                                     // AC_RND_CONV, false
ieeeef32 r4 = ac_math::ac_sub_dp<AC_TRN_ZERO, true>(a1, a2);

bf16 b1 = 2.0, b2 = 1.5;
bf16 r5 = ac_math::ac_add_dp (b1, b2); //add, default parameters
                                     //AC_TRN_ZERO, false
bf16 r6 = ac_math::ac_add_dp <AC_RND_CONV, false>(b1, b2);
```

3.3.1. The structure of a dual-path adder

The dual-path addition function is supposed to provide a more delay efficient addition for the standard datatypes of ac_std_float.h. In the meantime, the area of this design will be increased compared to the add function of ac_std_float.h.

The dual-path adder is designed as two parallel paths, the N and the R path. The N path is selected when an effective subtraction is computed, and the difference of the exponents is less than 2 and the mantissa result needs more than one bit of normalization. In any other case the R path is selected. Following these principles, the alignment of the two addends in path N is simplified to a single bit shifting, and the normalization in path R is similarly simplified. Also, path N has no need for rounding.

Because of the two parallel paths, the area of the operator is increased, however the simplified paths reduce the delay of the design.